

```

1: // === articulation.cpp ===
2: int n; // number of nodes
3: vector<vector<int>> adj; // adjacency list of graph
4: vector<bool> visited;
5: vector<int> tin, low;
6: int timer;
7:
8: void dfs(int v, int p = -1) {
9:     visited[v] = true;
10:    tin[v] = low[v] = timer++;
11:    int children=0;
12:    for (int to : adj[v]) {
13:        if (to == p) continue;
14:        if (visited[to]) {
15:            low[v] = min(low[v], tin[to]);
16:        } else {
17:            dfs(to, v);
18:            low[v] = min(low[v], low[to]);
19:            if (low[to] >= tin[v] && p!=-1)
20:                IS_CUTPOINT(v);
21:            ++children;
22:        }
23:    }
24:    if(p == -1 && children > 1)
25:        IS_CUTPOINT(v);
26: }
27:
28: void find_cutpoints() {
29:     timer = 0;
30:     visited.assign(n, false);
31:     tin.assign(n, -1);
32:     low.assign(n, -1);
33:     for (int i = 0; i < n; ++i) {
34:         if (!visited[i])
35:             dfs(i);
36:     }
37: }
38: // === binpow.cpp ===
39: const int mod = 998244353;
40: int binpow(int a, int b, long long m = mod){
41:     a %= m;
42:     int res = 1;
43:     while (b > 0){
44:         if (b & 1) res = res * a % m;
45:         a = a * a % m;
46:         b >>= 1;
47:     }
48:     return res%m;
49: }
50: // binpow(number,mod-2,mod) gives mod inverse if mod is prime
51: // === bridges.cpp ===
52: void IS_BRIDGE(int v,int to); // some function to process the found bridge
53: int n; // number of nodes
54: vector<vector<int>> adj; // adjacency list of graph
55: vector<bool> visited;
56: vector<int> tin, low;
57: int timer;
58:
59: void dfs(int v, int p = -1) {
60:     visited[v] = true;
61:     tin[v] = low[v] = timer++;
62:     bool parent_skipped = false;
63:     for (int to : adj[v]) {
64:         if (to == p && !parent_skipped) {
65:             parent_skipped = true;
66:             continue;
67:         }
68:         if (visited[to]) {
69:             low[v] = min(low[v], tin[to]);

```

```

70:         } else {
71:             dfs(to, v);
72:             low[v] = min(low[v], low[to]);
73:             if (low[to] > tin[v])
74:                 IS_BRIDGE(v, to);
75:         }
76:     }
77: }
78:
79: void find_bridges() {
80:     timer = 0;
81:     visited.assign(n, false);
82:     tin.assign(n, -1);
83:     low.assign(n, -1);
84:     for (int i = 0; i < n; ++i) {
85:         if (!visited[i])
86:             dfs(i);
87:     }
88: }
89: // === combinatorics.cpp ===
90: template<typename T, const int P>
91: class combinatorics{
92:     //combinatorics<mint,2> c(200);
93:     //2 is the number whose power will be precomputed
94: public:
95:     int n;
96:     vector<T> inv, fac, ifac, pw;
97:     combinatorics (int n) : n(n), inv(n+1), fac(n+1), ifac(n+1), pw(n+1){
98:         fac[0] = inv[0] = ifac[0] = pw[0] = T(1);
99:
100:        for(int i = 1; i <= n; i ++){
101:            inv[i] = T(1)/T(i), fac[i] = fac[i - 1] * T(i), ifac[i] = ifac[i - 1]
* inv[i], pw[i] = pw[i - 1] * T(P);
102:        }
103:
104:        T ncr(int n, int r){
105:            if(n < r or r < 0)
106:                return 0;
107:            return fac[n] * ifac[r] * ifac[n - r];
108:        }
109:    };
110: // === dsu.cpp ===
111: const int N = 3e5;
112: int parent[N],sz[N];
113:
114: void make_set(int v) {
115:     parent[v] = v;
116:     sz[v] = 1;
117: }
118: int find_set(int v) {
119:     if (v == parent[v])
120:         return v;
121:     return parent[v] = find_set(parent[v]);
122: }
123: void union_sets(int a, int b) {
124:     a = find_set(a);
125:     b = find_set(b);
126:     if (a != b) {
127:         if (sz[a] < sz[b])
128:             swap(a, b);
129:         parent[b] = a;
130:         sz[a] += sz[b];
131:     }
132: }
133: // === egcd.cpp ===
134: // ax + by = gcd
135: int egcd(int a, int b, int& x, int& y) {
136:     x = 1, y = 0;
137:     int x1 = 0, y1 = 1, a1 = a, b1 = b;

```

```

138:     while (b1) {
139:         int q = a1 / b1;
140:         tie(x, x1) = make_tuple(x1, x - q * x1);
141:         tie(y, y1) = make_tuple(y1, y - q * y1);
142:         tie(a1, b1) = make_tuple(b1, a1 - q * b1);
143:     }
144:     return a1; //gcd
145: }
146:
147: // if gcd(a,m) = 1;
148: // ax + my = 1 => x is the mod inverse of a with mod m
149:
150: int modinv(int a, int m) {
151:     int x, y;
152:     int g = egcd(a, m, x, y);
153:     if (g == 1) return (x % m + m) % m;
154:     else return -1;
155: }
156: // === fenwick_tree.cpp ===
157: struct Fenwick {
158:     vector<int> bit;
159:     int n;
160:     Fenwick(int n) {
161:         this->n = n + 1;
162:         bit.assign(n + 1, 0);
163:     }
164:     Fenwick(vector<int> a)
165:         : Fenwick(a.size()) {
166:         for (size_t i = 0; i < a.size(); i++)
167:             add(i, a[i]);
168:     }
169:     int sum(int idx) {
170:         int ret = 0;
171:         for (++idx; idx > 0; idx -= idx & -idx)
172:             ret += bit[idx];
173:         return ret;
174:     }
175:     int sum(int l, int r) {
176:         return sum(r) - sum(l - 1);
177:     }
178:     void add(int idx, int delta) {
179:         for (++idx; idx < n; idx += idx & -idx)
180:             bit[idx] += delta;
181:     }
182: };
183: // === lazysegtree.cpp ===
184: // MODIFY THIS SECTION, UPDATE SECTION and PUSH FUNCTION
185: struct Node {
186:     long long sum;
187:     long long max_val;
188:     long long sz;
189:
190:     // Identity Element
191:     static Node neutral() {
192:         return {0, (long long)-4e18, 0};
193:     }
194:
195:     // Leaf Initialization: How to create a node from a single array value
196:     static Node from_value(long long v) {
197:         return {v, v, 1};
198:     }
199: };
200:
201: // MERGE
202: Node merge(const Node& a, const Node& b) {
203:     // If one side is empty/neutral (size 0), return the other
204:     if (a.sz == 0) return b;
205:     if (b.sz == 0) return a;
206:

```

```

207:     return {
208:         a.sum + b.sum,
209:         max(a.max_val, b.max_val),
210:         a.sz + b.sz
211:     };
212: }
213:
214: // TAG
215: struct Tag {
216:     long long add;
217:
218:     // Identity Tag
219:     static Tag neutral() {
220:         return {0};
221:     }
222: };
223:
224: // APPLY TAG
225: void apply_tag(Node& node, const Tag& tag) {
226:     if (node.sz == 0) return;
227:     node.sum += tag.add * node.sz; // Range Add logic for Sum
228:     node.max_val += tag.add;      // Range Add logic for Max
229: }
230:
231: // COMBINE TAGS: How to merge a new tag onto an existing one
232: void combine_tags(Tag& old_tag, const Tag& new_tag) {
233:     old_tag.add += new_tag.add;
234: }
235:
236: // CHECK (Descent): Predicate for finding an index
237: // returns first index where check is true
238: bool check(const Node& node, long long k) {
239:     return node.max_val >= k;
240: }
241:
242: struct LazySegTree {
243:     int n;
244:     vector<Node> tree;
245:     vector<Tag> lazy;
246:
247:     LazySegTree(const vector<int>& a) {
248:         n = a.size();
249:         tree.resize(4 * n, Node::neutral());
250:         lazy.resize(4 * n, Tag::neutral());
251:
252:         build(a, 1, 0, n - 1);
253:     }
254:
255:     void push(int node) {
256:
257:         //MODIFY THIS
258:         if (lazy[node].add == 0) return;
259:
260:         apply_tag(tree[2 * node], lazy[node]);
261:         combine_tags(lazy[2 * node], lazy[node]);
262:
263:         apply_tag(tree[2 * node + 1], lazy[node]);
264:         combine_tags(lazy[2 * node + 1], lazy[node]);
265:
266:         lazy[node] = Tag::neutral();
267:     }
268:
269:     void build(const vector<int>& a, int node, int start, int end) {
270:         if (start == end) {
271:             tree[node] = Node::from_value(a[start]);
272:         } else {
273:             int mid = (start + end) / 2;
274:             build(a, 2 * node, start, mid);
275:             build(a, 2 * node + 1, mid + 1, end);

```

```

276:         tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
277:     }
278: }
279:
280: void update(int l, int r, long long val, int node, int start, int end) {
281:     if (start > end || start > r || end < l) return;
282:
283:     if (start >= l && end <= r) {
284:
285:         //MODIFY THIS
286:         Tag new_tag = {val};
287:
288:         apply_tag(tree[node], new_tag);
289:         combine_tags(lazy[node], new_tag);
290:         return;
291:     }
292:
293:     push(node);
294:     int mid = (start + end) / 2;
295:     update(l, r, val, 2 * node, start, mid);
296:     update(l, r, val, 2 * node + 1, mid + 1, end);
297:     tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
298: }
299:
300: Node query(int l, int r, int node, int start, int end) {
301:     if (start > end || start > r || end < l) return Node::neutral();
302:     if (start >= l && end <= r) return tree[node];
303:
304:     push(node);
305:     int mid = (start + end) / 2;
306:     return merge(query(l, r, 2 * node, start, mid),
307:                 query(l, r, 2 * node + 1, mid + 1, end));
308: }
309:
310: // Descent: Find first index in [l, r] satisfying check()
311: int find_first(int l, int r, long long k, int node, int start, int end) {
312:
313:     if (start > end || start > r || end < l || !check(tree[node], k)) {
314:         return -1;
315:     }
316:
317:     if (start == end) return start;
318:
319:     push(node);
320:     int mid = (start + end) / 2;
321:
322:     int res = find_first(l, r, k, 2 * node, start, mid);
323:
324:     if (res == -1) {
325:         res = find_first(l, r, k, 2 * node + 1, mid + 1, end);
326:     }
327:
328:     return res;
329: }
330:
331: void build(const vector<int>& a) { build(a, 1, 0, n - 1); }
332: void update(int l, int r, long long val) { update(l, r, val, 1, 0, n - 1); }
333: Node query(int l, int r) { return query(l, r, 1, 0, n - 1); }
334: int find_first(int l, int r, long long k) { return find_first(l, r, k, 1, 0, n
- 1); }
335: };
336: //LazySegTree st(arr);
337: // === linear_sieve.cpp ===
338: const int N = 10000000;
339: vector<int> lp(N+1); //lowest prime factor
340: vector<int> pr; // prime list
341: void sieve(){
342:     for(int i = 2; i <= N; ++i){
343:         if (lp[i] == 0) {

```

```

344:             lp[i] = i;
345:             pr.push_back(i);
346:         }
347:         for (int j = 0; i * pr[j] <= N; ++j) {
348:             lp[i * pr[j]] = pr[j];
349:             if (pr[j] == lp[i]) {
350:                 break;
351:             }
352:         }
353:     }
354: }
355:
356: // === modint.cpp ===
357: #include<bits/stdc++.h>
358: using namespace std;
359:
360: #define int int
361: template<const int &MOD>
362: struct _m_int {
363:     int val;
364:
365:     _m_int(int64_t v = 0) {
366:         if (v < 0) v = v % MOD + MOD;
367:         if (v >= MOD) v %= MOD;
368:         val = int(v);
369:     }
370:
371:     _m_int(uint64_t v) {
372:         if (v >= MOD) v %= MOD;
373:         val = int(v);
374:     }
375:
376:     _m_int(int v) : _m_int(int64_t(v)) {}
377:     _m_int(unsigned v) : _m_int(uint64_t(v)) {}
378:
379:     explicit operator int() const { return val; }
380:     explicit operator unsigned() const { return val; }
381:     explicit operator int64_t() const { return val; }
382:     explicit operator uint64_t() const { return val; }
383:     explicit operator double() const { return val; }
384:     explicit operator long double() const { return val; }
385:
386:     _m_int& operator+=(const _m_int &other) {
387:         val -= MOD - other.val;
388:         if (val < 0) val += MOD;
389:         return *this;
390:     }
391:
392:     _m_int& operator-=(const _m_int &other) {
393:         val -= other.val;
394:         if (val < 0) val += MOD;
395:         return *this;
396:     }
397:
398:     static unsigned fast_mod(uint64_t x, unsigned m = MOD) {
399:     #if !defined(_WIN32) || defined(_WIN64)
400:         return unsigned(x % m);
401:     #endif
402:         // Optimized mod for Codeforces 32-bit machines.
403:         // x must be less than 2^32 * m for this to work, so that x / m fits in an
unsigned 32-bit int.
404:         unsigned x_high = unsigned(x >> 32), x_low = unsigned(x);
405:         unsigned quot, rem;
406:         asm("divl %4\n"
407:             : "=a" (quot), "=d" (rem)
408:             : "d" (x_high), "a" (x_low), "r" (m));
409:         return rem;
410:     }
411:

```

```

412:     _m_int& operator*=(const _m_int &other) {
413:         val = fast_mod(uint64_t(val) * other.val);
414:         return *this;
415:     }
416:
417:     _m_int& operator/=(const _m_int &other) {
418:         return *this *= other.inv();
419:     }
420:
421: friend _m_int operator+(const _m_int &a, const _m_int &b) { return _m_int(a) +
= b; }
422: friend _m_int operator-(const _m_int &a, const _m_int &b) { return _m_int(a) -
= b; }
423: friend _m_int operator*(const _m_int &a, const _m_int &b) { return _m_int(a) *
= b; }
424: friend _m_int operator/(const _m_int &a, const _m_int &b) { return _m_int(a) /
= b; }
425:
426:     _m_int& operator++() {
427:         val = val == MOD - 1 ? 0 : val + 1;
428:         return *this;
429:     }
430:
431:     _m_int& operator--() {
432:         val = val == 0 ? MOD - 1 : val - 1;
433:         return *this;
434:     }
435:
436:     _m_int operator++(int) { _m_int before = *this; ++*this; return before; }
437:     _m_int operator--(int) { _m_int before = *this; --*this; return before; }
438:
439:     _m_int operator-() const {
440:         return val == 0 ? 0 : MOD - val;
441:     }
442:
443: friend bool operator==(const _m_int &a, const _m_int &b) { return a.val == b.v
al; }
444: friend bool operator!=(const _m_int &a, const _m_int &b) { return a.val != b.v
al; }
445: friend bool operator<(const _m_int &a, const _m_int &b) { return a.val < b.val
; }
446: friend bool operator>(const _m_int &a, const _m_int &b) { return a.val > b.val
; }
447: friend bool operator<=(const _m_int &a, const _m_int &b) { return a.val <= b.v
al; }
448: friend bool operator>=(const _m_int &a, const _m_int &b) { return a.val >= b.v
al; }
449:
450: static const int SAVE_INV = int(1e6) + 5;
451: static _m_int save_inv[SAVE_INV];
452:
453: static void prepare_inv() {
454:     // Ensures that MOD is prime, which is necessary for the inverse algorithm
below.
455:     for (int64_t p = 2; p * p <= MOD; p += p % 2 + 1)
456:         assert(MOD % p != 0);
457:
458:     save_inv[0] = 0;
459:     save_inv[1] = 1;
460:
461:     for (int i = 2; i < SAVE_INV; i++)
462:         save_inv[i] = save_inv[MOD % i] * (MOD - MOD / i);
463: }
464:
465:     _m_int inv() const {
466:         if (save_inv[1] == 0)
467:             prepare_inv();
468:
469:         if (val < SAVE_INV)

```

```

470:         return save_inv[val];
471:
472:     _m_int product = 1;
473:     int v = val;
474:
475:     do {
476:         product *= MOD - MOD / v;
477:         v = MOD % v;
478:     } while (v >= SAVE_INV);
479:
480:     return product * save_inv[v];
481: }
482:
483:     _m_int pow(int64_t p) const {
484:         if (p < 0)
485:             return inv().pow(-p);
486:
487:         _m_int a = *this, result = 1;
488:
489:         while (p > 0) {
490:             if (p & 1)
491:                 result *= a;
492:
493:             p >>= 1;
494:
495:             if (p > 0)
496:                 a *= a;
497:
498:         }
499:
500:         return result;
501:
502: friend ostream& operator<<(ostream &os, const _m_int &m) {
503:     return os << m.val;
504: }
505: };
506:
507: template<const int &MOD> _m_int<MOD> _m_int<MOD>::save_inv[_m_int<MOD>::SAVE_INV];
508:
509: const int MOD = ;
510: using mint = _m_int<MOD>;
511: void __print(mint x) { cerr << x; }
512: // === multiple_string_hashes.cpp ===
513: /////////////// !!!!!!!!!!!!!!! 1-based indexing !!!!!!!!!!!!!!! ///////////////
514: mt19937_64 RNG(chrono::steady_clock::now().time_since_epoch().count());
515: struct Hash{
516:     int b, n; // b = number of hashes ; n = length of string
517:     const int mod = 1e9 + 7;
518:     vector<vector<int>> fw, bc, pb, ib;
519:     vector<int> bases;
520:
521:     inline int power(int x, int y){
522:         if (y == 0){
523:             return 1;
524:         }
525:
526:         int v = power(x, y / 2);
527:         v = 1LL * v * v % mod;
528:         if (y & 1) return 1LL * v * x % mod;
529:         else return v;
530:     }
531:
532:     inline void init(int nn, int bb, string str){
533:         n = nn;
534:         b = bb;
535:         fw = vector<vector<int>>(b, vector<int>(n + 2, 0));
536:         bc = vector<vector<int>>(b, vector<int>(n + 2, 0));
537:         pb = vector<vector<int>>(b, vector<int>(n + 2, 1));
538:         ib = vector<vector<int>>(b, vector<int>(n + 2, 1));

```

```

539:     bases = vector<int>(b);
540:     str = "0" + str;
541:
542:     for (auto &x : bases) x = RNG() % (mod / 10);
543:
544:     for (int i = 0; i < b; i++){
545:         for (int j = 1; j <= n + 1; j++){
546:             pb[i][j] = 1LL * pb[i][j - 1] * bases[i] % mod;
547:         }
548:         ib[i][n + 1] = power(pb[i][n + 1], mod - 2);
549:         for (int j = n; j >= 1; j--){
550:             ib[i][j] = 1LL * ib[i][j + 1] * bases[i] % mod;
551:         }
552:
553:         for (int j = 1; j <= n; j++){
554:             fw[i][j] = (fw[i][j - 1] + 1LL * (str[j] - 'a' + 1) * pb[i][j]) %
mod;
555:         }
556:         for (int j = n; j >= 1; j--){
557:             bc[i][j] = (bc[i][j + 1] + 1LL * (str[j] - 'a' + 1) * pb[i][n + 1
- j]) % mod;
558:         }
559:     }
560: }
561:
562: inline int getfwhash(int l, int r, int i){
563:     int ans = fw[i][r] - fw[i][l - 1];
564:     ans = 1LL * ans * ib[i][l - 1] % mod;
565:
566:     if (ans < 0) ans += mod;
567:
568:     return ans;
569: }
570:
571: inline int getbchash(int l, int r, int i){
572:     int ans = bc[i][l] - bc[i][r + 1];
573:     ans = 1LL * ans * ib[i][n - r] % mod;
574:
575:     if (ans < 0) ans += mod;
576:
577:     return ans;
578: }
579:
580: inline bool equal(int l1, int r1, int l2, int r2){
581:     for (int i = 0; i < b; i++){
582:         int v1, v2;
583:         if (l1 <= r1) v1 = getfwhash(l1, r1, i);
584:         else v1 = getbchash(r1, l1, i);
585:
586:         if (l2 <= r2) v2 = getfwhash(l2, r2, i);
587:         else v2 = getbchash(r2, l2, i);
588:
589:         if (v1 != v2) return false;
590:     }
591:     return true;
592: }
593:
594: inline bool pal(int l, int r){
595:     return equal(l, r, r, l);
596: }
597: };
598:
599: // Hash h;
600: // h.init(n,2,str); (length,no of hashes,string)
601: // === ordered_set.cpp ===
602: #include <ext/pb_ds/assoc_container.hpp>
603: #include <ext/pb_ds/tree_policy.hpp>
604: using namespace __gnu_pbds;
605: template<class T> using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tre

```

```

e_order_statistics_node_update>;
606: template<class T> using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tr
ee_tag, tree_order_statistics_node_update>;
607:
608: /** X.find_by_order(k) return kth element. 0 indexed.
609: /** X.order_of_key(k) returns count of elements strictly less than k.
610:
611:
612:
613: // === segmented_sieve.cpp ===
614: void solve(){
615:     int l,r;
616:     cin>>l>>r;
617:     int lim = sqrt(r)+1;
618:
619:     vector<int> base_primes;
620:     vector<bool> is_prime(lim+1,1);
621:     for(int i=2;i<=r;i++){
622:         if(!is_prime[i]) continue;
623:         for(int j = i*i;j<=lim;j+=i) {
624:             // dbg(j);
625:             is_prime[j] = false;
626:         }
627:     }
628:
629:     for(int i = 2;i<=r;i++) if(is_prime[i]) base_primes.push_back(i);
630:
631:     vector<int> is_prime_seg(r-l+1,1);
632:     for(int p : base_primes){
633:         int st = ((l+p-1)/p) * p;
634:         for(int j = st;j<=r;j+=p) is_prime_seg[j-l] = p;
635:     }
636:     if(l==1) is_prime_seg[0] = false;
637:     // is_prime_seg[i] = true ==> l+i is prime
638: }
639: // === segment_tree.cpp ===
640: // MODIFY THIS SECTION, BUILD and UPDATE
641: struct Node {
642:     long long sum;
643:     long long max_val;
644:
645:     static Node neutral() {
646:         return {0, (long long)-1e18};
647:     }
648: };
649: // Merge logic
650: Node merge(const Node& a, const Node& b) {
651:     return {
652:         a.sum + b.sum,
653:         max(a.max_val, b.max_val)
654:     };
655: }
656: // Descent Predicate
657: // we get first index from the left where the check function is true
658: bool check(const Node& node, long long k) {
659:     return node.max_val >= k;
660: }
661: struct SegTree {
662:     int n;
663:     vector<Node> tree;
664:
665:     SegTree(const vector<int>& a) {
666:         n = a.size();
667:         tree.resize(4 * n, Node::neutral());
668:
669:         build(a,1,0,n-1);
670:     }
671:
672:     void build(const vector<int>& a, int node, int start, int end) {

```

```

673:         if (start == end) {
674:
675:             // MODIFY THIS
676:             // MODIFY THIS
677:             // MODIFY THIS
678:             tree[node] = { (long long)a[start], (long long)a[start] };
679:
680:         } else {
681:             int mid = (start + end) / 2;
682:             build(a, 2 * node, start, mid);
683:             build(a, 2 * node + 1, mid + 1, end);
684:             tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
685:         }
686:     }
687:
688: void update(int idx, int val, int node, int start, int end) {
689:     if (start == end) {
690:
691:         // MODIFY THIS
692:         // MODIFY THIS
693:         // MODIFY THIS
694:         tree[node] = { (long long)val, (long long)val };
695:
696:     } else {
697:         int mid = (start + end) / 2;
698:         if (start <= idx && idx <= mid) {
699:             update(idx, val, 2 * node, start, mid);
700:         } else {
701:             update(idx, val, 2 * node + 1, mid + 1, end);
702:         }
703:         tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
704:     }
705: }
706:
707: Node query(int l, int r, int node, int start, int end) {
708:     if (r < start || end < l) {
709:         return Node::neutral();
710:     }
711:     if (l <= start && end <= r) {
712:         return tree[node];
713:     }
714:     int mid = (start + end) / 2;
715:     Node p1 = query(l, r, 2 * node, start, mid);
716:     Node p2 = query(l, r, 2 * node + 1, mid + 1, end);
717:     return merge(p1, p2);
718: }
719:
720: // Descent: Find first index in range [l, r] where check returns true
721: int find_first(int l, int r, long long k, int node, int start, int end) {
722:     if (start > r || end < l || !check(tree[node], k)) {
723:         return -1;
724:     }
725:
726:     if (start == end) {
727:         return start;
728:     }
729:
730:     int mid = (start + end) / 2;
731:     int res = -1;
732:
733:     // 2. Try Left
734:     if (mid >= l) {
735:         res = find_first(l, r, k, 2 * node, start, mid);
736:     }
737:
738:     // 3. Try Right if not found in Left
739:     if (res == -1) {
740:         res = find_first(l, r, k, 2 * node + 1, mid + 1, end);
741:     }

```

```

742:
743:         return res;
744:     }
745:
746:     // Wrappers
747:     void build(const vector<int>& a) { build(a, 1, 0, n - 1); }
748:     void update(int idx, int val) { update(idx, val, 1, 0, n - 1); }
749:     Node query(int l, int r) { return query(l, r, 1, 0, n - 1); }
750:     int find_first(int l, int r, long long k) { return find_first(l, r, k, 1, 0, n
- 1); }
751: };
752: // SegTree st(arr);
753: // === sieve.cpp ===
754: const int MAXN = 10000000; // adjust as needed
755: vector<int> spf(MAXN+1); // smallest prime factor
756: vector<int> primelist;
757:
758: // Precompute SPF in O(n log log n)
759: void sieve() {
760:     iota(spf.begin(), spf.end(), 0); // spf[i] = i initially
761:     for (int i = 2; i * i <= MAXN; i++) {
762:         if (spf[i] == i) { // i is prime
763:             for (int j = i * i; j <= MAXN; j += i) {
764:                 if (spf[j] == j) spf[j] = i;
765:             }
766:         }
767:     }
768:     for (int i = 2; i <= MAXN; i++) {
769:         if (spf[i] == i) {
770:             primelist.push_back(i);
771:         }
772:     }
773: }
774: // prime factorization in O(log n)
775: vector<pair<int,int>> getPrimeFactors(int x) {
776:     vector<pair<int,int>> res;
777:     while (x > 1) {
778:         int p = spf[x], cnt = 0;
779:         while (x % p == 0) {
780:             x /= p;
781:             cnt++;
782:         }
783:         res.push_back({p, cnt});
784:     }
785:     return res;
786: }
787: // all divisors in O(log n + #divisors(n))
788: // #divisors of n = n ^ 1/3
789: vector<int> getDivisors(int x) {
790:     auto pf = getPrimeFactors(x);
791:     vector<int> divisors = {1};
792:
793:     for (auto [p, cnt] : pf) {
794:         int sz = divisors.size();
795:         long long pw = 1;
796:         for (int i = 1; i <= cnt; i++) {
797:             pw *= p;
798:             for (int j = 0; j < sz; j++) {
799:                 divisors.push_back(divisors[j] * pw);
800:             }
801:         }
802:     }
803:     sort(divisors.begin(), divisors.end());
804:     return divisors;
805: }
806:
807:
808: // === sparse_table.cpp ===
809: int log2_floor(unsigned long i) {

```

```

810:     return i ? __builtin_clzll(1) - __builtin_clzll(i) : -1;
811: }
812:
813: signed main() {
814:
815:     vector<int> array = {1,5,3,1,5};
816:
817:     int MAXN = 2*1e5, K = log2_floor(MAXN), N=5;
818:     int st[K + 1][MAXN];
819:
820:     copy(array.begin(), array.end(), st[0]);
821:
822:     for (int i = 1; i <= K; i++)
823:         for (int j = 0; j + (1 << i) <= N; j++){
824:             // st[i][j] = f(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
825:             // f is the function for range query
826:
827:             // st[i][j] = st[i - 1][j] + st[i - 1][j + (1 << (i - 1))]; --> Precompute
for Range sum queries
828:             st[i][j] = min(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]); // range min
queries
829:         }
830:
831:         // Range sum queries --> O(log N)
832:         // long long sum = 0;
833:         // for (int i = K; i >= 0; i--) {
834:         //     if ((1 << i) <= R - L + 1) {
835:         //         sum += st[i][L];
836:         //         L += 1 << i;
837:         //     }
838:         // }
839:
840:
841:         // Range min query --> O(1)
842:         int L = 1, R=4;
843:         int i = log2_floor(R - L + 1);
844:         int minimum = min(st[i][L], st[i][R - (1 << i) + 1]);
845:
846:         cout<<minimum<<endl;
847:
848:         return 0;
849:
850: }
851: // === trie_bit_array.cpp ===
852: // faster than normal trie but not memeroy efficinet as there is no deletions
853: const int MAX_BITS = 31;
854: struct TrieNode {
855:     int next[2] = {-1, -1};
856:     int count = 0;
857: };
858: struct BitTrie {
859:     vector<TrieNode> trie;
860:
861:     BitTrie() {
862:         trie.emplace_back(); // root node at index 0
863:     }
864:     void insert(int num) {
865:         int node = 0;
866:         for (int i = MAX_BITS; i >= 0; --i) {
867:             int bit = (num >> i) & 1;
868:             if (trie[node].next[bit] == -1) {
869:                 trie[node].next[bit] = trie.size();
870:                 trie.emplace_back();
871:             }
872:             node = trie[node].next[bit];
873:             trie[node].count++;
874:         }
875:     }
876:     int maxXor(int num) {

```

```

877:         int node = 0, result = 0;
878:         for (int i = MAX_BITS; i >= 0; --i) {
879:             int bit = (num >> i) & 1;
880:             int opposite = bit ^ 1;
881:             if (trie[node].next[opposite] != -1 &&
882:                 trie[trie[node].next[opposite]].count > 0) {
883:                 result |= (1 << i);
884:                 node = trie[node].next[opposite];
885:             } else if (trie[node].next[bit] != -1) {
886:                 node = trie[node].next[bit];
887:             } else {
888:                 break;
889:             }
890:         }
891:         return result;
892:     }
893: };
894:
895: // === trie_bit.cpp ===
896: struct BitTrieNode {
897:     BitTrieNode* child[2];
898:     int count;
899:     BitTrieNode() {
900:         child[0] = child[1] = nullptr;
901:         count = 0;
902:     }
903: };
904: class BitTrie {
905: private:
906:     BitTrieNode* root;
907:     static const int BIT_LEN = 31;
908: public:
909:     BitTrie() {
910:         root = new BitTrieNode();
911:     }
912:
913:     void insert(int x) {
914:         BitTrieNode* node = root;
915:         node->count++;
916:         for (int b = BIT_LEN - 1; b >= 0; b--) {
917:             int bit = (x >> b) & 1;
918:             if (!node->child[bit]) {
919:                 node->child[bit] = new BitTrieNode();
920:             }
921:             node = node->child[bit];
922:             node->count++;
923:         }
924:     }
925:
926:     void remove(int x) {
927:         if (!contains(x)) {
928:             return;
929:         }
930:         BitTrieNode* node = root;
931:         node->count--;
932:         for (int b = BIT_LEN - 1; b >= 0; b--) {
933:             int bit = (x >> b) & 1;
934:             BitTrieNode* nxt = node->child[bit];
935:             nxt->count--;
936:             node = nxt;
937:         }
938:     }
939:     bool contains(int x) const {
940:         BitTrieNode* node = root;
941:         for (int b = BIT_LEN - 1; b >= 0; b--) {
942:             if (!node) return false;
943:             int bit = (x >> b) & 1;
944:             if (!node->child[bit] || node->child[bit]->count == 0) {
945:                 return false;

```

```

946:         }
947:         node = node->child[bit];
948:     }
949:     return node && node->count > 0;
950: }
951: int maxXor(int x) const {
952:     BitTrieNode* node = root;
953:     int result = 0;
954:     for (int b = BIT_LEN - 1; b >= 0; b--) {
955:         int bit = (x >> b) & 1;
956:         int desired = bit ^ 1;
957:         if (node->child[desired] && node->child[desired]->count > 0) {
958:             result |= (1 << b);
959:             node = node->child[desired];
960:         } else if (node->child[bit] && node->child[bit]->count > 0) {
961:             node = node->child[bit];
962:         } else {
963:             break;
964:         }
965:     }
966:     return result;
967: }
968: };
969: // BitTrie trie;
970: // === trie.cpp ===
971: struct TrieNode {
972:     TrieNode* children[26];
973:     bool is_end;
974:     int prefix_count;
975:     int end_count;
976:     TrieNode() {
977:         is_end = false;
978:         prefix_count = 0;
979:         end_count = 0;
980:         for (int i = 0; i < 26; i++) {
981:             children[i] = nullptr;
982:         }
983:     }
984: };
985:
986: class Trie {
987: private:
988:     TrieNode* root;
989:
990: public:
991:     Trie() {
992:         root = new TrieNode();
993:     }
994:
995:     void insert(const string &word) {
996:         TrieNode* node = root;
997:         node->prefix_count++;
998:         for (char ch : word) {
999:             int idx = ch - 'a';
1000:             if (idx < 0 || idx >= 26) {
1001:                 continue;
1002:             }
1003:             if (!node->children[idx]) {
1004:                 node->children[idx] = new TrieNode();
1005:             }
1006:             node = node->children[idx];
1007:             node->prefix_count++;
1008:         }
1009:         node->is_end = true;
1010:         node->end_count++;
1011:     }
1012:
1013:     bool search(const string &word) const {
1014:         TrieNode* node = root;

```

```

1015:         for (char ch : word) {
1016:             int idx = ch - 'a';
1017:             if (idx < 0 || idx >= 26 || !node->children[idx]) {
1018:                 return false;
1019:             }
1020:             node = node->children[idx];
1021:         }
1022:         return node->is_end;
1023:     }
1024:     bool startsWith(const string &prefix) const {
1025:         TrieNode* node = root;
1026:         for (char ch : prefix) {
1027:             int idx = ch - 'a';
1028:             if (idx < 0 || idx >= 26 || !node->children[idx]) {
1029:                 return false;
1030:             }
1031:             node = node->children[idx];
1032:         }
1033:         return true;
1034:     }
1035:     int countWordsEqualTo(const string &word) const {
1036:         TrieNode* node = root;
1037:         for (char ch : word) {
1038:             int idx = ch - 'a';
1039:             if (idx < 0 || idx >= 26 || !node->children[idx]) {
1040:                 return 0;
1041:             }
1042:             node = node->children[idx];
1043:         }
1044:         return node->end_count;
1045:     }
1046:     int countWordsStartingWith(const string &prefix) const {
1047:         TrieNode* node = root;
1048:         for (char ch : prefix) {
1049:             int idx = ch - 'a';
1050:             if (idx < 0 || idx >= 26 || !node->children[idx]) {
1051:                 return 0;
1052:             }
1053:             node = node->children[idx];
1054:         }
1055:         return node->prefix_count;
1056:     }
1057:     void remove(const string &word) {
1058:         TrieNode* node = root;
1059:         vector<TrieNode*> path;
1060:         path.reserve(word.size() + 1);
1061:         path.push_back(root);
1062:
1063:         for (char ch : word) {
1064:             int idx = ch - 'a';
1065:             if (idx < 0 || idx >= 26 || node->children[idx] == nullptr) {
1066:                 return;
1067:             }
1068:             node = node->children[idx];
1069:             path.push_back(node);
1070:         }
1071:         if (node->end_count == 0) return;
1072:
1073:         node->end_count--;
1074:         if (node->end_count == 0) {
1075:             node->is_end = false;
1076:         }
1077:         for (TrieNode* nd : path) {
1078:             nd->prefix_count--;
1079:         }
1080:
1081:     }
1082: };
1083: // Trie trie;

```

```

1084:
1085: // === unordered_map_custom_hash.cpp ===
1086: struct custom_hash {
1087:     static uint64_t splitmix64(uint64_t x) {
1088:         x += 0x9e3779b97f4a7c15;
1089:         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
1090:         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
1091:         return x ^ (x >> 31);
1092:     }
1093:     size_t operator()(uint64_t x) const {
1094:         static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
1095:         return splitmix64(x + FIXED_RANDOM);
1096:     }
1097: };
1098: // === xorbasis.cpp ===
1099: template<typename T = long long>
1100: struct XorBasis {
1101:     static const int B = 62;
1102:     T basis[B];
1103:     int sz = 0;
1104:
1105:     XorBasis() {
1106:         memset(basis, 0, sizeof(basis));
1107:         sz = 0;
1108:     }
1109:
1110:     void clear() {
1111:         memset(basis, 0, sizeof(basis));
1112:         sz = 0;
1113:     }
1114:     void insert(T x) {
1115:         for (int i = B - 1; i >= 0; i--) {
1116:             if ((x >> i) & 1) {
1117:                 if (!basis[i]) {
1118:                     basis[i] = x;
1119:                     sz++;
1120:                     return;
1121:                 }
1122:                 x ^= basis[i];
1123:             }
1124:         }
1125:     }
1126:
1127:     // Is there any subset having xor = x
1128:     bool contains(T x) {
1129:         for (int i = B - 1; i >= 0; i--) {
1130:             if ((x >> i) & 1) {
1131:                 if (!basis[i]) return false;
1132:                 x ^= basis[i];
1133:             }
1134:         }
1135:         return x == 0;
1136:     }
1137:
1138:     // Max subset xor after xoring starting it with x
1139:     T max_xor(T x = 0) {
1140:         for (int i = B - 1; i >= 0; i--) {
1141:             x = max(x, x ^ basis[i]);
1142:         }
1143:         return x;
1144:     }
1145:
1146:     // Min subset xor after xoring starting it with x
1147:     T min_xor(T x = 0) {
1148:         for (int i = B - 1; i >= 0; i--) {
1149:             x = min(x, x ^ basis[i]);
1150:         }
1151:         return x;

```

```

1152:     }
1153:
1154:     // Kth smallest subset xor (1-based index)
1155:     // 1st smallest is 0. Returns -1 if k > 2^sz.
1156:     T kth_smallest(T k) {
1157:         T total = (T)1 << sz;
1158:         if (k < 1 || k > total) return -1;
1159:
1160:         T res = 0;
1161:         T cnt = total;
1162:
1163:         // Iterate MSB to LSB
1164:         for (int i = B - 1; i >= 0; i--) {
1165:             if (basis[i]) {
1166:                 cnt >>= 1;
1167:                 if (k > cnt) {
1168:                     if (!(res >> i) & 1) res ^= basis[i];
1169:                     k -= cnt;
1170:                 }
1171:                 else {
1172:                     if ((res >> i) & 1) res ^= basis[i];
1173:                 }
1174:             }
1175:         }
1176:         return res;
1177:     }
1178:
1179:     // Kth largest subset xor
1180:     T kth_largest(T k) {
1181:         T total = (T)1 << sz;
1182:         if (k < 1 || k > total) return -1;
1183:         return kth_smallest(total - k + 1);
1184:     }
1185:
1186:     // Number of subsets having xor < x
1187:     T count_less(T x) {
1188:         if (x < 0) return 0;
1189:
1190:         T res = 0;
1191:         T cnt = (T)1 << sz;
1192:         T current_path = 0;
1193:
1194:         for (int i = B - 1; i >= 0; i--) {
1195:             if (basis[i]) {
1196:                 cnt >>= 1;
1197:                 int bit_x = (x >> i) & 1;
1198:                 int bit_path = (current_path >> i) & 1;
1199:
1200:                 if (bit_x == 1) {
1201:                     res += cnt;
1202:                     if (bit_path == 0) current_path ^= basis[i];
1203:                 } else {
1204:                     if (bit_path == 1) current_path ^= basis[i];
1205:                 }
1206:             } else {
1207:                 int bit_x = (x >> i) & 1;
1208:                 int bit_path = (current_path >> i) & 1;
1209:
1210:                 if (bit_path != bit_x) {
1211:                     if (bit_path < bit_x) return res + cnt;
1212:                     else return res;
1213:                 }
1214:             }
1215:         }
1216:         return res;
1217:     }
1218: };
1219: // XorBasis<long long> xb;
1220:

```